Tarea Practica p4 Iker Núñez Meana UO306425

Para empezar, se pedía implementar el algoritmo en Java:

```java
public class ColoreoGrafo {

    public static Map<String, String> realizarVoraz(Map<String, List<String>> grafo){
        String[] colours = {"red", "blue", "green",
        "yellow", "orange", "purple", "cyan", "magenta", "lime"};
        Map<String,String> result = new HashMap<String,String>();

        List<String> vecinos;
        HashSet<String> coloursUsed;

        for(String nodo : grafo.keySet()){
            vecinos = grafo.get(nodo);
            coloursUsed = new HashSet<String>();

            for(String nodoVecino : vecinos){
                if(result.containsKey(nodoVecino)){
                    coloursUsed.add(result.get(nodoVecino));
                }
            }

            for(int j = 0; j < colours.length; j++){
                if(!coloursUsed.contains(colours[j])){
                    result.put(nodo, colours[j]);
                    break;
                }
            }
        }

        return result;
    }
}
```

El heurístico que escogí fue usar el primer color disponible de la lista que no este usado por ninguno de los nodos adyacentes al que se mira.

Tras contrastar los resultados, fueron correctos.

La complejidad de este algoritmo parece cuadrática, ya que hay 2 bucles for anidados. Sin embargo, en el bucle for que recorre los vecinos, nunca se va a recorrer todo el grafo, solo el subgrafo que forman los vecinos del nodo. Por esta razón, la complejidad se aproxima a O(n).

```java
public class DevoradorTiempos {
    Run | Debug
    public static void main(String[] args){
        String[] ficheros = {"g4.json", "g8.json", "g16.json", "g32.json", "g64.json", "g128.json",
            "g256.json","g512.json","g1024.json","g2048.json","g4096.json","g8192.json",
            "g16384.json", "g32768.json","g65536.json"};

        long t1;
        long t2;
        double tiempo;
        for(String fichero : ficheros){
            t1 = System.currentTimeMillis();

            Devorador.main(new String[]{fichero});

            t2 = System.currentTimeMillis();

            tiempo = t2 - t1;

            System.out.printf(format: "Fichero: %s ; Tiempo: %f", fichero, tiempo);
            System.out.println();
        }
    }
}
```

los tiempos que salen, serían correspondientes a una complejidad aproximadamente de O(n):

n	tColoreado(ms)
4	27
8	2
16	3
32	5
64	3
128	4
256	6
512	12
1024	35
2048	23
4096	32
8192	39
16384	89
32768	167
65536	294

Si analizamos el algoritmo, vemos que para cada nodo, se recorren sus vecinos y después el array de colores. Pero claro, los nodos no tienen muchos vecinos, y en cuanto a los colores, para las matrices usadas no se usan todos los colores, luego los fors internos no se ejecutan completamente. Esta podría ser una explicación para que los tiempos obtenidos no se correspondan con la complejidad del algoritmo.

Después, se pide la implementación en Python que sería la siguiente:

```python
def realizar_voraz(grafo):
    colours = ["red", "blue", "green",
        "yellow", "orange", "purple", "cyan", "magenta", "lime"]

    result = {}
    for nodo in grafo:
        vecinos = grafo[nodo]
        coloursUsed = []

        for i in range(len(vecinos)):
            nodoVecino = vecinos[i]

            coloursUsed = {result[str(v)] for v in vecinos if str(v) in result}

        for j in range(len(colours)):
            if(not(colours[j] in coloursUsed)):
                result[nodo] = colours[j]
                break

    return result
```

El heurístico es el mismo que en la implementación de Java y el resultado también es correcto. En cuanto a la complejidad, sucede lo mismo, midiendo los tiempos con esta clase:

```python
def main():
    ficheros = ["g4.json", "g8.json", "g16.json", "g32.json", "g64.json", "g128.json",
            "g256.json","g512.json","g1024.json","g2048.json","g4096.json","g8192.json",
            "g16384.json", "g32768.json","g65536.json"]

    t1 = 0
    t2 = 0
    tiempo = 0

    for fichero in ficheros:
        nombreFichero = "sols/"+fichero
        with open(nombreFichero) as f:
            mapa = json.load(f)
            f.close()

        t1 = time()

        aux.realizar_voraz(mapa["grafo"])

        t2 = time()

        tiempo = (t2 - t1)*1000

        print("Fichero: ", fichero, " tiempo: ",tiempo )
        print("\n")
```

se obtienen los siguientes tiempos:

n	tColoreado(ms)
4	0,0000
8	0,0000
16	0,0000
32	0,0000
64	0,0000
128	0,0000
256	0,0000
512	0,0000
1024	0,0000
2048	16,3910
4096	14,7933
8192	25,9991
16384	37,7975
32768	207,6943
65536	324,4348

Tras analizar los tiempos obtenidos en ambos casos, estos se corresponden con la complejidad, si bien en el caso de Python son menos precisos.